

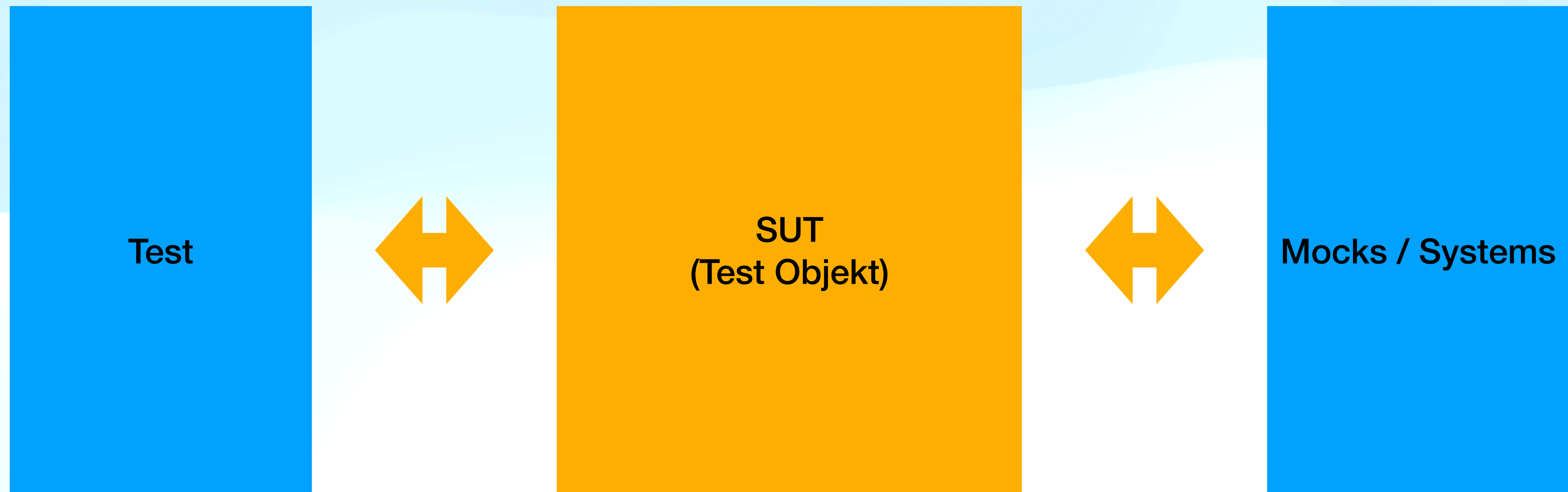


Software Tests

Warum brauchen wir Software Tests?

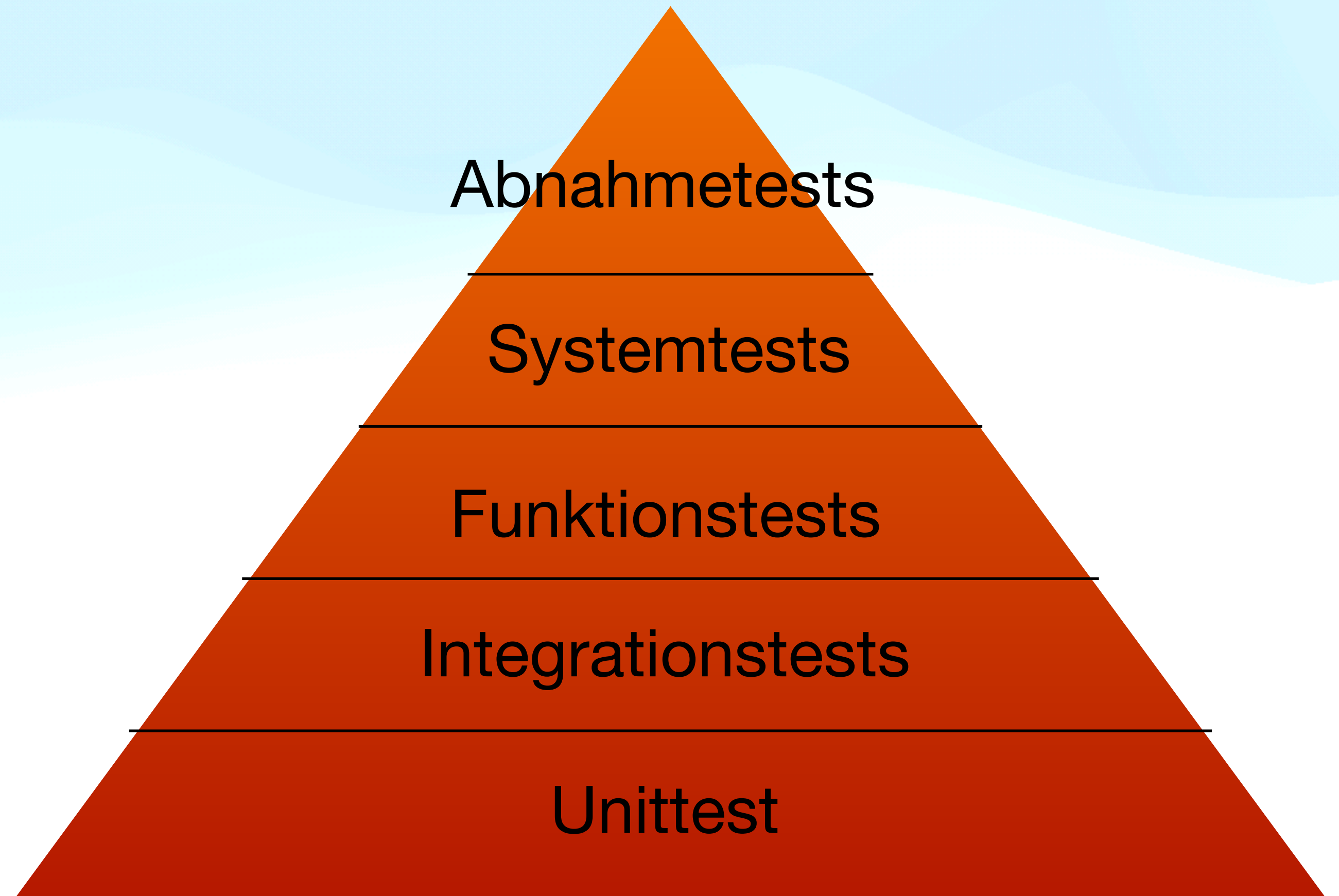
- Je umfangreicher Software ist, desto höher ist die Wahrscheinlichkeit, bei einer Codeanpassung auch Bugs einzubauen
- Qualitätsmerkmal
- Weniger Fehlerberichte
- Höhere Kundenzufriedenheit
- Ermöglicht Refactoring
- Tests sind auch eine Art der Dokumentation

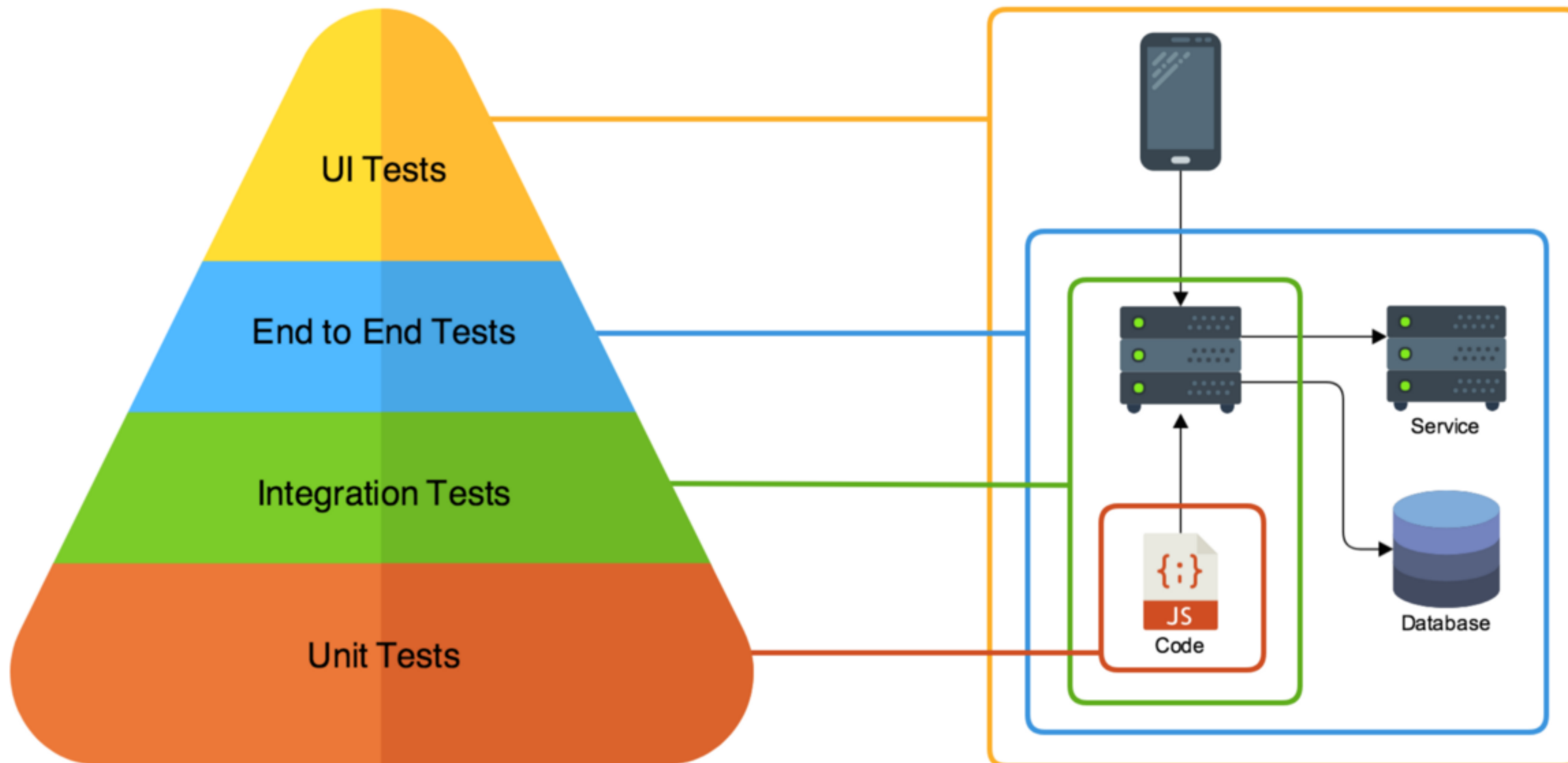
System Under Test (SUT)



Arten von Software Tests

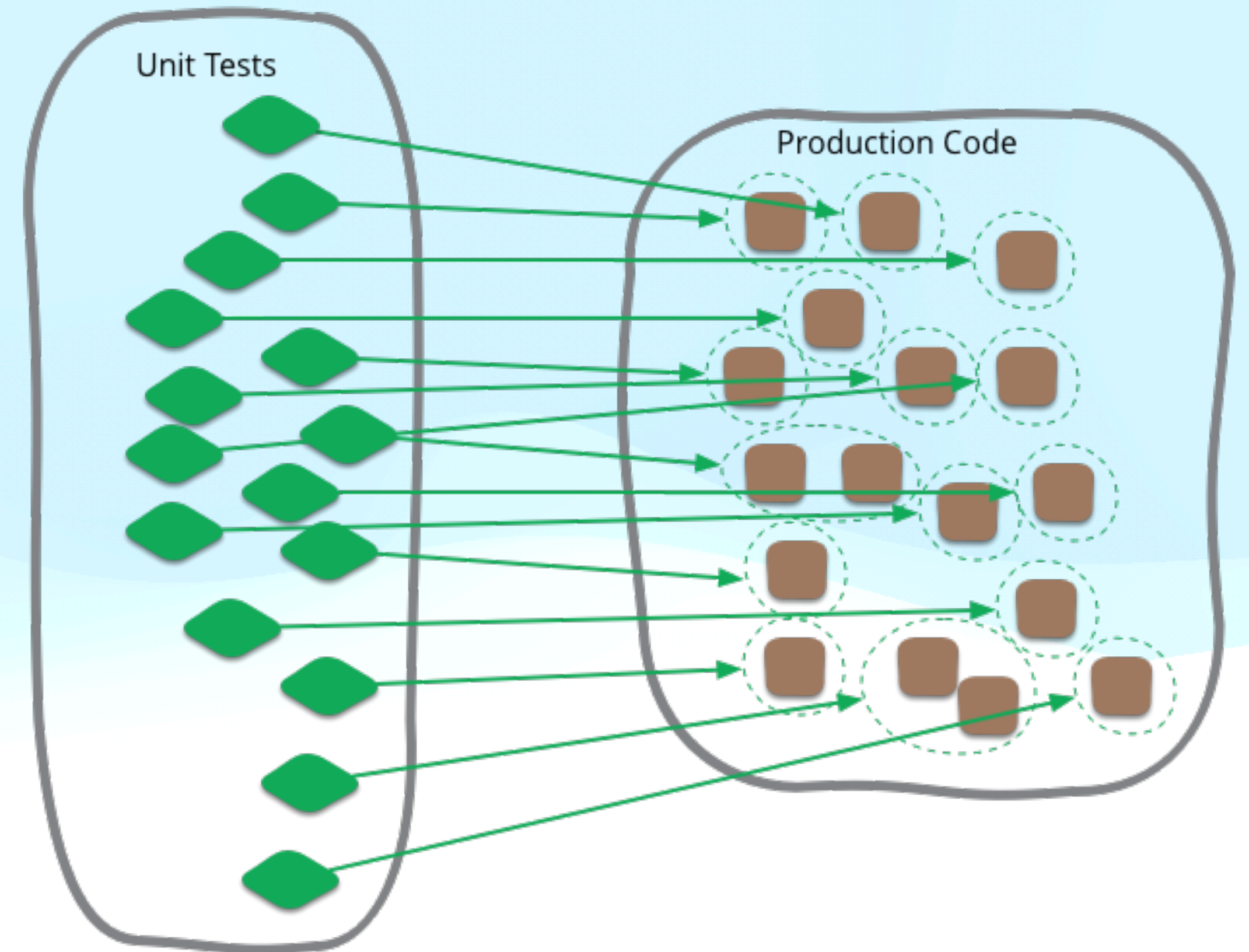
- Unittests
- Integrationstests
- Funktionstests
- Systemtests
 - End-To-End Tests (e2e tests)
 - Akzeptanztests
 - Lasttests / Performanztests
 - Smoketests



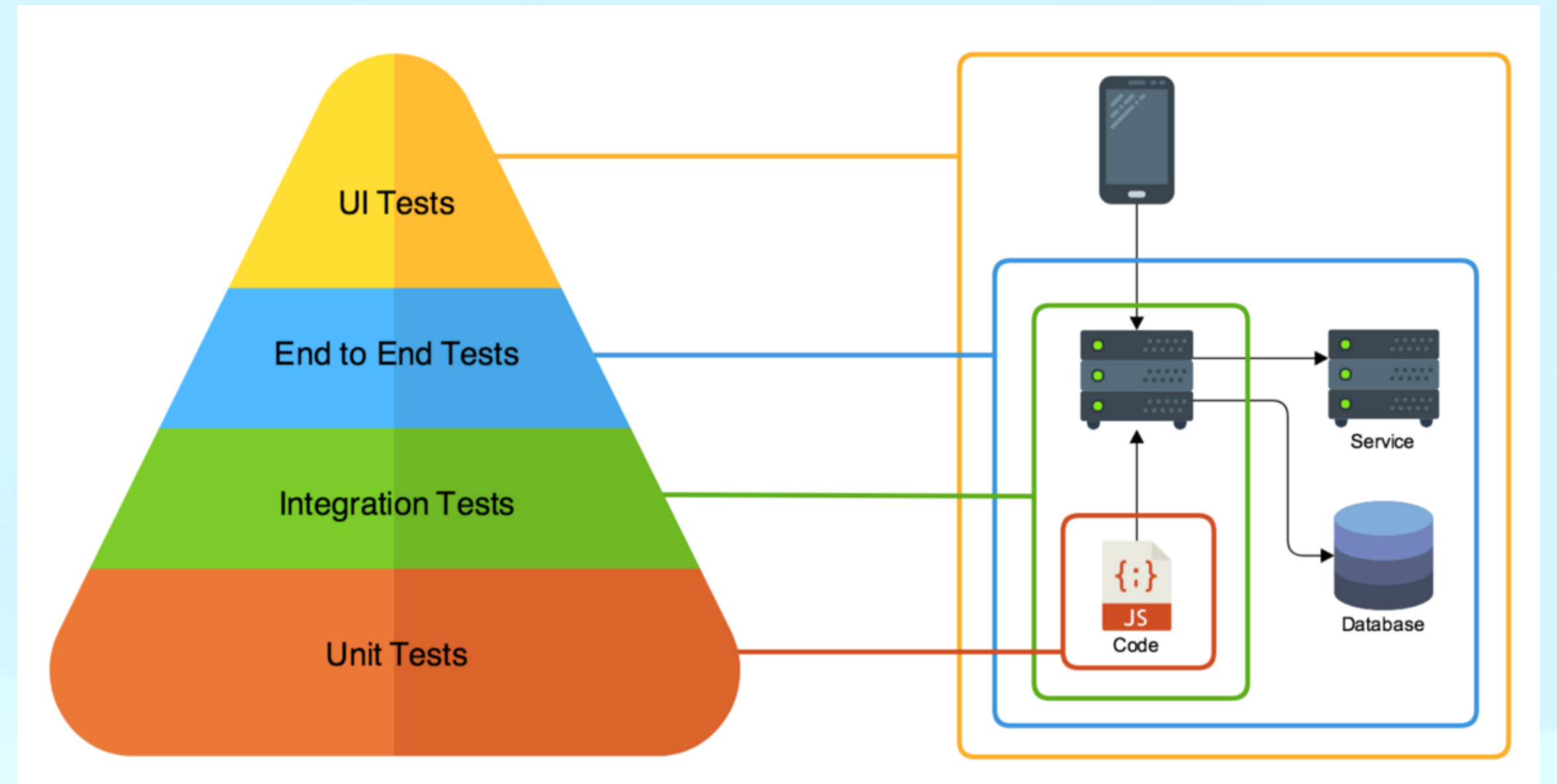


Unittests

- Testen einzelne Funktionen oder Methoden von Klassen, Modulen oder Komponenten
- Low-Level Tests
- Einfach zu automatisieren
- Schnell auszuführen
- Möglichst viel Produktivcode sollte mit Unittests abgetastet sein
- Dürfen den internen Zustand des SUT verifizieren und manipulieren



Integrationstests

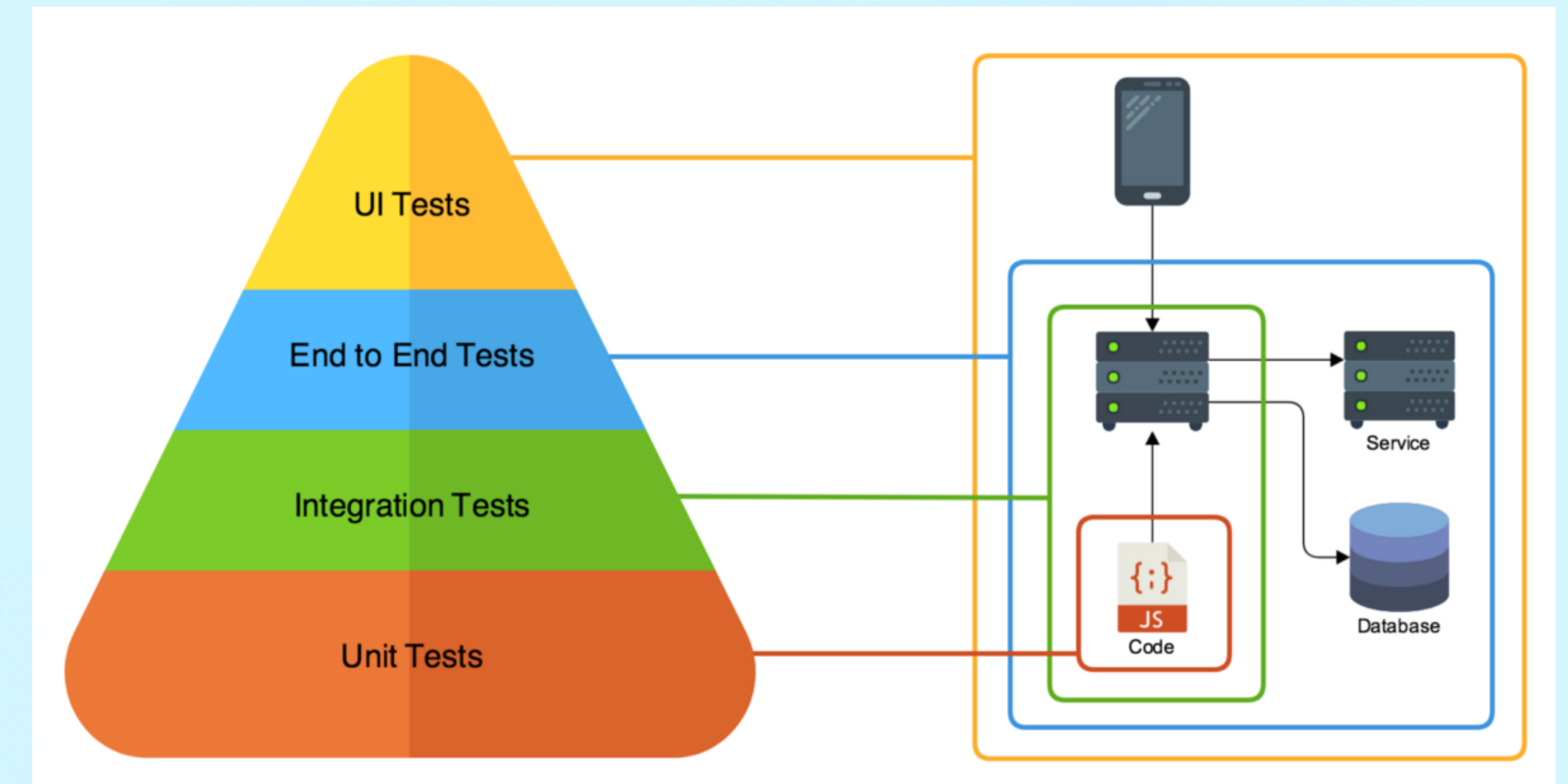


- Testen das Zusammenspiel von verschiedenen Modulen oder Komponenten
- Beispiel: Datenbankinteraktion, Mikroservice Kommunikation
- Teurer in der Ausführung, da mehrere Programmteile gestartet werden müssen
- Sie können auch den internen Zustand einer Anwendung prüfen

Funktionstests

- Testen Business Logik / Anforderungen
- Unterschied zu Integrationstests:
 - Integrationstests testen die Lauffähigkeit eines SUT
 - Funktionstests prüfen die zurückgegebenen Werte / Ergebnisse

End-To-End Tests



- E2E Tests ahmen den End-Benutzer nach
- Die komplette Anwendung muss hochgefahren werden
- Teuer in der Ausführung
- Anforderungen wie auch Regressionen können getestet werden
- Wartbarkeit ist erschwert
- Verwenden nur öffentliche Schnittstellen

Akzeptanztests

- Prüfen an die Anwendung gestellte KPIs
- Aufbau analog zu E2E Tests
- Können automatisiert und manuell ausgeführt werden

Lasttests / Performanztests

- Prüfen wie sich eine Anwendung unter Belastung verhält
- Geben eine Rückmeldung über
 - Zuverlässigkeit
 - Geschwindigkeit
 - Reaktivität
 - Skalierbarkeit
- Beispiele: Tests mit vielen Daten oder mit vielen Anfragen pro Zeit

Smoketests

- Testen einige Hauptfunktionen der Anwendung
- Einsatzzweck:
 - Zur Vorabprüfung vor teuren Integration-/e2e-tests
 - Prüfung, ob die Anwendung prinzipiell in einer neuen Umgebung lauffähig ist

Regressionstests

- Verhindern das erneute Einschleichen bereits behobener Fehler
- Anwendbar auf allen Testarten



Test Driven Development (TDD)

Code - Test - Design

Red - Green - Refactor

Die Aktivitäten des TDD

- Einen Unittest schreiben (sollte fehlschlagen)
- „Gerade genug“ Code schreiben, dass der Test gut geht
- Code-Refactoring, bis er den „Regeln der Einfachheit“ entspricht



Regeln der Einfachheit

1. Der Code wird durch (erfolgreiche) Unittests verifiziert
2. Es existieren keine Code Duplikate
3. Der Code ist hat genau eine Verantwortlichkeit
4. Es wurden so wenig Elemente wie möglich verwendet (um Regeln 1-3 zu erfüllen)

Warum TDD?

- Code ist oft schwer testbar, wenn Tests danach geschrieben werden
- Oft werden Tests von Außenstehenden als unnötige Investition erachtet
- Tests im Nachgang testen manchmal auch die Bugs mit
- Bei der Entwicklung mit TDD können 45% aller Fehler vermieden werden
 - Beim Einsatz von Unittests ohne TDD: 30%

Akzeptanztestgetriebene Entwicklung (BDD)

- Anforderungen werden in Form von Systemtests definiert
- Code wird nicht gegen schriftlich formulierte Anforderungen entwickelt
- Code wird gegen Systemtests entwickelt
- Tests müssen nicht unbedingt automatisiert sein
- Tests müssen auch von Nicht-Entwicklern gelesen werden können